

Carmen Fuenzalida
Johan Muñoz

DIAGRAMAS UML

I.

Se realizaron principalmente dos tipos de diagramas:

- **Diagramas de secuencia**

Representan la interacción entre los actores y el sistema para distintos casos de uso (prácticas, tesis, proyectos, becados, noticias, búsquedas y filtros). En ellos se modela el flujo entre Visitante → Frontend → API → Base de datos, mostrando cómo se obtienen y despliegan los datos.

- **Diagrama de arquitectura**

Este diagrama muestra una visión global del sistema, incluyendo:

- Frontend (usuario visitante y administrador)
- Backend (API en Node.js con sus módulos: usuarios, proyectos, noticias, prácticas, etc.)
- Estructura interna (routes, controllers, services, models)
- Base de datos (MongoDB)
- Servicios externos (Google Drive)
- Modelo de datos (índices como usuarios, prácticas, becados, proyectos)

El diagrama que mejor permite comunicar la arquitectura del sistema es el diagrama de arquitectura, ya que entrega una visión global de cómo se organiza y se estructura la solución. En este se identifican claramente los distintos elementos del sistema, como el frontend, el backend, la base de datos y los servicios externos, así como sus relaciones e interacciones. Además, permite comprender la distribución en capas y la organización interna del backend, evidenciando la separación en módulos como routes, controllers, services y models.

II. DIAGRAMA DE SECUENCIA

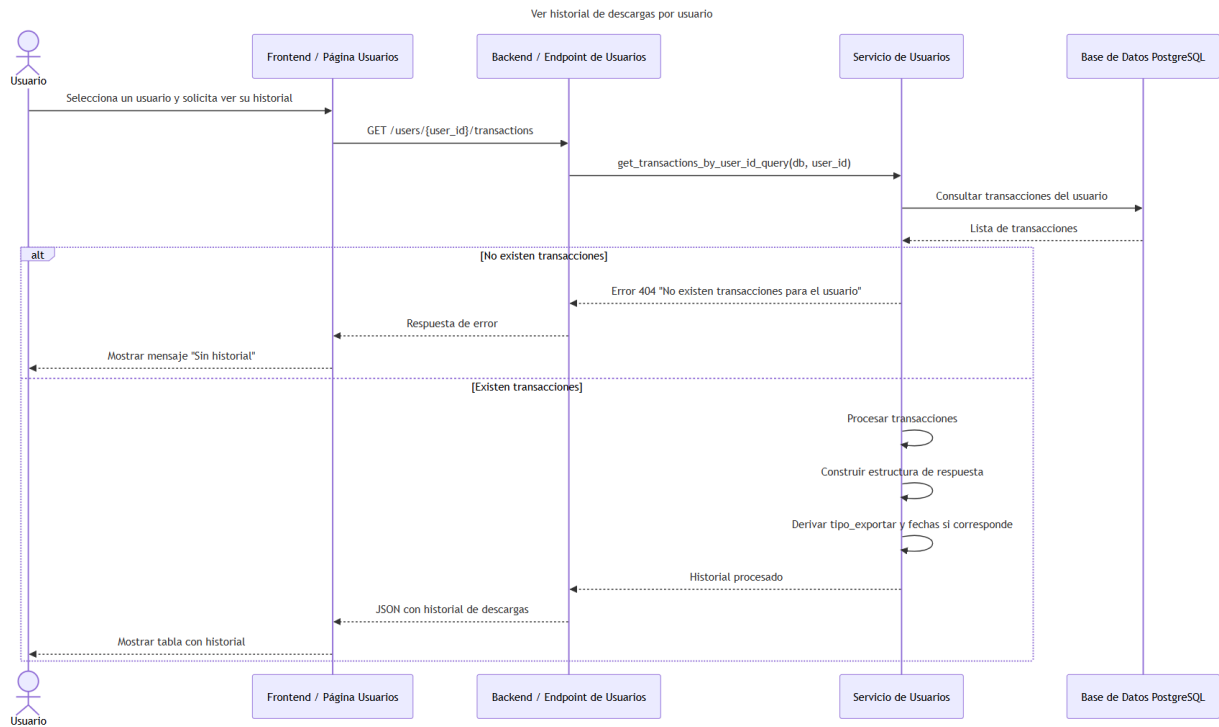


DIAGRAMA DE SECUENCIA ORIGINAL

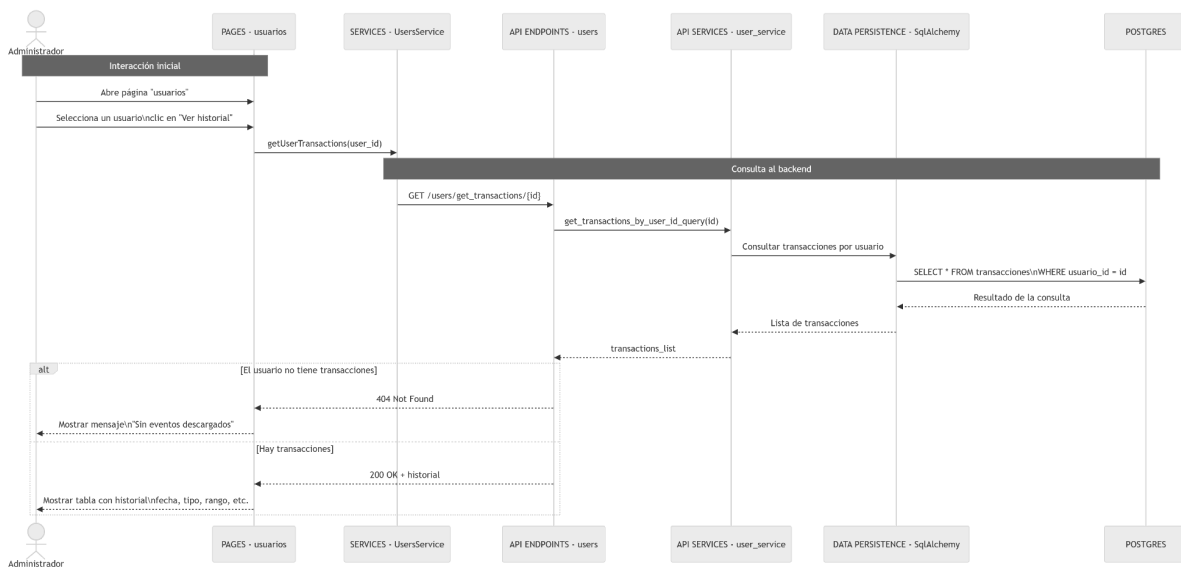


DIAGRAMA DE COMPONENTES

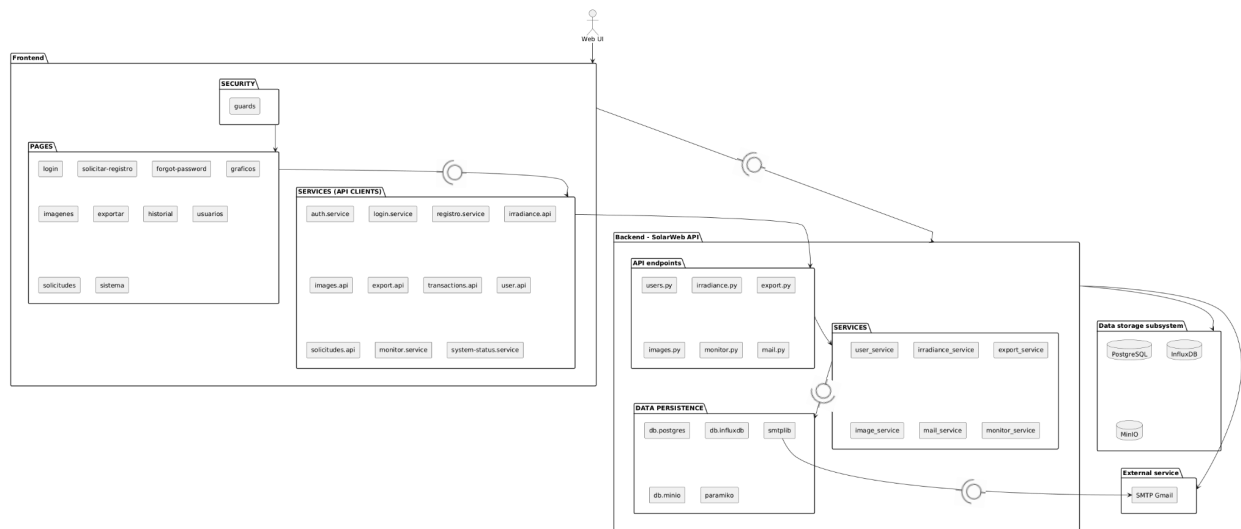
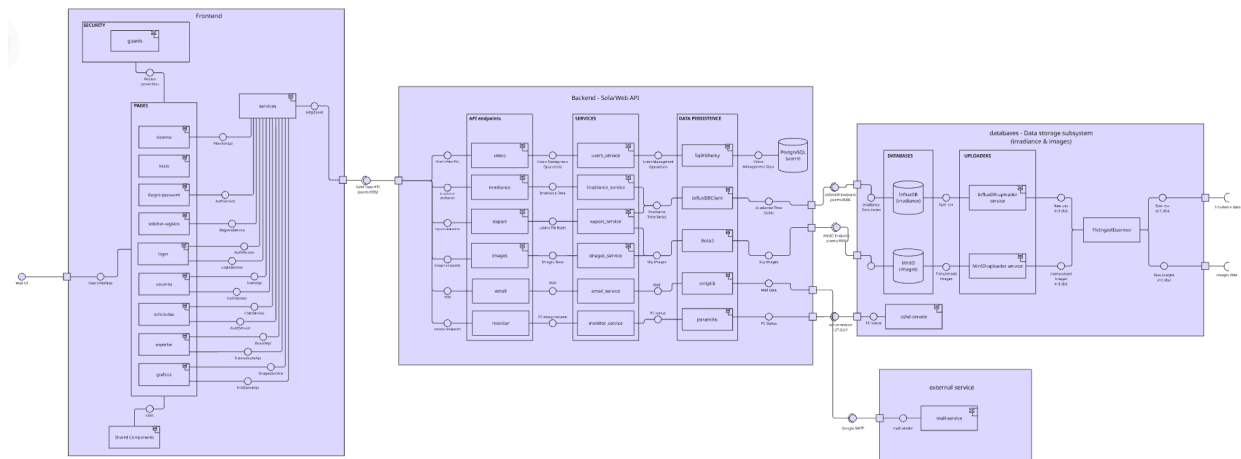


DIAGRAMA DE COMPONENTES ORIGINAL



III.

El **diagrama de secuencia** propuesto reproduce correctamente el flujo principal del diagrama original para la consulta del historial de descargas por usuario, manteniendo la misma secuencia de interacción entre frontend, backend, servicio y base de datos. Ambos consideran los dos escenarios (existencia o no de transacciones) y reflejan adecuadamente la separación por capas del sistema. Sin embargo, el diagrama desarrollado simplifica la estructura al agrupar componentes que en el original aparecen más detallados (como API Services y Data Persistence).

Por otra parte, el diagrama propuesto incorpora un mayor nivel de detalle en la lógica interna del servicio, mostrando explícitamente el procesamiento de las transacciones y la construcción de la respuesta, aspectos que en el original se mantienen implícitos. En contraste, el diagrama original enfatiza elementos más técnicos, como el uso de códigos HTTP específicos y consultas SQL, lo que lo hace más cercano a la implementación. El diagrama desarrollado tiene un equilibrio adecuado entre claridad y detalle, siendo más comprensible desde una perspectiva funcional sin perder coherencia con el diseño original.

El **diagrama de componentes** original presenta un alto nivel de detalle, mostrando explícitamente las interacciones entre componentes individuales como páginas, servicios, endpoints y capas de persistencia. Esta representación permite comprender con precisión el flujo de datos y las dependencias específicas del sistema, pero genera una visualización compleja, con múltiples conexiones y cruces que dificultan su lectura.

En contraste, el diagrama propuesto adopta un enfoque más abstracto y organizado, agrupando los elementos en bloques funcionales como frontend, backend, persistencia y servicios externos, y priorizando las conexiones entre estos bloques en lugar de entre componentes individuales. Esto mejora significativamente la legibilidad y permite entender de manera más rápida las responsabilidades de cada subsistema y la arquitectura general. Aunque se pierde cierto nivel de detalle técnico en comparación con el original, se posee más claridad, lo que lo hace más adecuado para una visión total del sistema.